

$$a_i = b_j = 1, \{a_{i-1} \sim a_1, b_{j-1} \sim b_1\} = \{0, 1\}$$

对于任-二进制的加法, $\overbrace{a_i a_{i-1} \dots a_1}^A + \overbrace{b_j b_{j-1} \dots b_1}^B = X$

简化条件: 将 $A \rightarrow A', B \rightarrow B'$ 使 $\text{len}(A') = \text{len}(B') = \max(\text{len}(A), \text{len}(B)) + 1$

不足处用0补齐, 易得 $\text{len}(X') = \text{len}(B') + 1$

$$(a_1, b_1) \Rightarrow \begin{array}{l} + (0, 0) \rightarrow (0, 0) \\ + (0, 1) \rightarrow (1, 0) \\ + (1, 0) \rightarrow (1, 0) \\ + (1, 1) \rightarrow (0, 1) \end{array}$$

此处可以统一为7数大值, 可以很快

即 $+(a_1, b_1) = (\text{NOR}(a_1, b_1), \text{AND}(a_1, b_1))$

前面补0的问题

本位 进位. (0不进位. 1进位.)

$$\text{同理 } +(a_2, b_2) = (\text{NOR}(a_2, b_2), \text{AND}(a_2, b_2))$$

$$(+ (a_2, b_2), \text{AND}(a_1, b_1)) \xrightarrow{\text{进位}(J)}$$

$$\begin{array}{l} J((0, 0), 0) \rightarrow (0, 0) \\ J((1, 0), 0) \rightarrow (1, 0) \\ J((0, 1), 0) \rightarrow (0, 1) \\ J((0, 0), 1) \rightarrow (1, 0) \\ J((1, 0), 1) \rightarrow (0, 1) \\ J((0, 1), 1) \rightarrow (1, 1) \end{array} \quad \begin{array}{l} \text{即 } J(+ (a_2, b_2), \text{AND}(a_1, b_1)) \\ = \left(\text{NOR}(+(a_1, b_2)[0], \text{AND}(a_1, b_1)), \right. \\ \left. \text{OR}(+(a_1, b_2)[1], \text{AND}(a_1, b_1)) \right) \end{array}$$

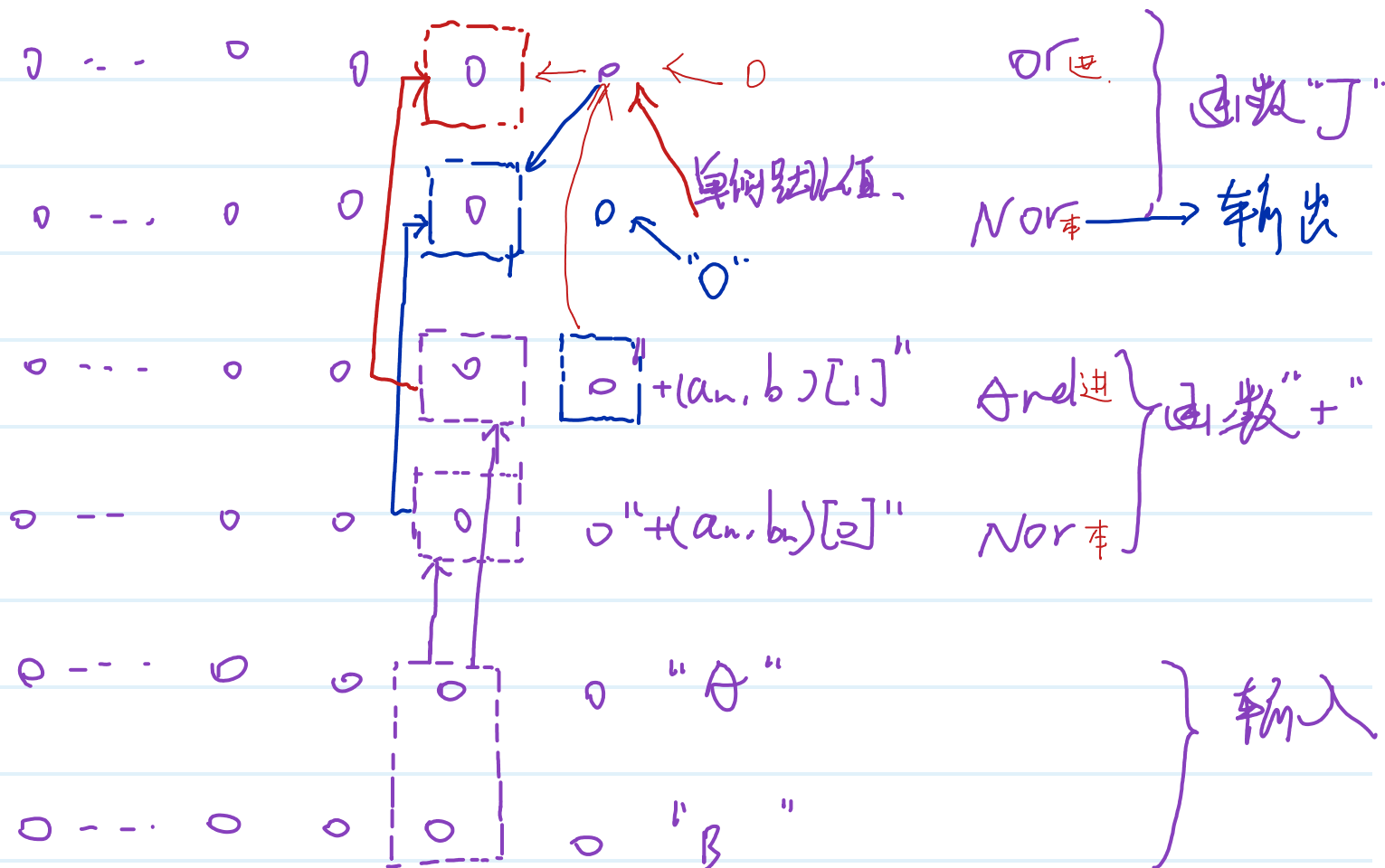
不妨设 $i \geq j$, 即 A 的位数大于 B.

对于第 n 位.

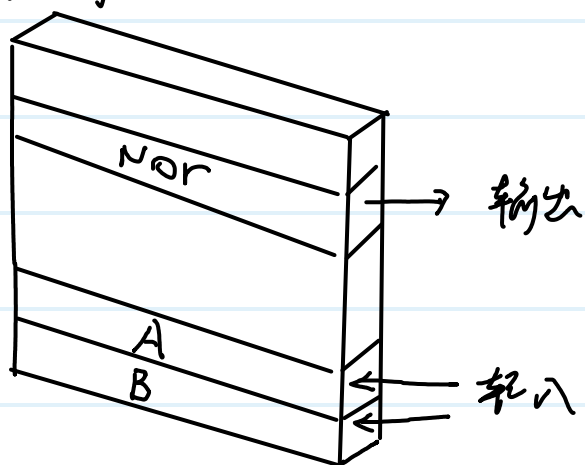
$$\begin{cases} + (a_n, b_n) = (\text{NOR}(a_n, b_n), \text{AND}(a_n, b_n)) & \textcircled{1} \\ J(+ (a_n, b_n), + (a_{n-1}, b_{n-1}) [1]) \\ = \left(\text{NOR}(\underbrace{+ (a_n, b_n) [0]}_{\neq}, \underbrace{+ (a_{n-1}, b_{n-1}) [1]}_{\neq}) \right. \\ \left. \text{OR}(\underbrace{+ (a_n, b_n) [1]}_{\neq}, \underbrace{+ (a_{n-1}, b_{n-1}) [1]}_{\neq}) \right) & \textcircled{2} \\ n=1 \text{ 时 } + (a_{n-1}, b_{n-1}) [1] = 0 \end{cases}$$

对于 X' , 即 $\overline{x_i x_{i-1} \dots x_1}$, 第 n 位 $x_n = J(+ (a_n, b_n), + (a_{n-1}, b_{n-1})) [0]$

阵列结构



加法阵列的简图表示:



减法:

1. 数域问题 $\rightarrow$ 负数定义 (我记得好像有利用第一位是 0/1 来定义正负的, 也记得补码可以定义负数. 但是具体怎么样需要推一下.)

$$a_i = b_j = 1, \quad \underbrace{\{a_{i-1} \sim a_1, b_{j-1} \sim b_1\}}_A = \underbrace{\{0, 1\}}_B$$

我打算画图推导一下
补码的使用了

对于位-二进制减法, $\underbrace{a_i a_{i-1} \dots a_1}_A - \underbrace{b_j b_{j-1} \dots b_1}_B = X$

$$\text{例如 } \begin{array}{r} 10000 \\ 168421 \end{array} - 1010 = 0110$$

$$16 - 10 = \boxed{6}$$

$$b = 2 + 4$$

$$\begin{array}{r} 1111 \\ - 1010 \\ \hline \end{array} = \boxed{0101}$$

$$A - B = A - (I_j - \text{not } B)$$

$$= A - I_j + \text{not } B$$

$$\begin{array}{r} 1011011 \\ - 11111 \\ \hline \end{array}$$

若 $A - I_j$ 是正数... $\rightarrow \checkmark$

$$= \begin{array}{r} 11100 \\ - \\ \hline \end{array}$$

$$\begin{array}{r} \textcircled{A} \\ - 14 \\ \hline \end{array}$$

$$\begin{array}{r} 1111 \\ 1011011 \\ - 0011111 \\ \hline \end{array}$$

后来我发现他好像推不出补码，应该以七真结果来定义负数。

几个设计原则：

1. 结构的固定性（适应性）：在一定的区间我给定的数字无论长短都可以用相同的程式去适应，让每个士兵的训练量少，训练内容复杂程度统一

2. 输入与输出的标准接口：比如让一个文书人员告诉一整排士兵在最开始如何进行举旗作为最初输入——那要是这样的话输出也要是这样的——暂且确定这种形式为标准输入输出吧

$$a_i = b_j = 1, \{a_{i-1} \sim a_i, b_{j-1} \sim b_j\} = \{0, 1\}$$

对于任意二进制加法, $\overbrace{a_i a_{i-1} \dots a_1}^A - \overbrace{b_j b_{j-1} \dots b_1}^B = X$

简化条件: 将 $A \rightarrow A', B \rightarrow B'$ 使 $\text{len}(A') = \text{len}(B') = \max(\text{len}(A), \text{len}(B))$
不足处用 0 补齐, 得 $\text{len}(X') = \text{len}(A')$

$$(a_1, b_1) \Rightarrow \begin{cases} (0, 0) \rightarrow (0, 0) \\ (0, 1) \rightarrow (1, 1) \\ (1, 0) \rightarrow (1, 0) \\ (1, 1) \rightarrow (0, 0) \end{cases} \quad \text{即 } -(a_1, b_1) = (\text{NOR}(a_1, b_1), 1)$$

$\text{TW}(a_1, b_1)$
TWI: 定义 $(0, 1) \rightarrow 1$
 $\text{not}(0, 1) \rightarrow 0$

本位 进位 (1代表进位, 0代表不进位)

$$\text{同理 } -(a_2, b_2) = (\text{NOR}(a_2, b_2), \text{TW}(a_2, b_2))$$

$$(-(a_2, b_2), \text{TW}(a_1, b_1)) \xRightarrow{\text{进位}(T)}$$

$$\begin{cases} T((0, 0), 0) \rightarrow (0, 0) \\ T((1, 0), 0) \rightarrow (1, 0) \\ T((1, 1), 0) \rightarrow (1, 1) \\ T((0, 0), 1) \rightarrow (1, 1) \\ T((1, 0), 1) \rightarrow (0, 0) \\ T((1, 1), 1) \rightarrow (0, 1) \end{cases} \quad \text{即 } T(-(a_2, b_2), \text{TW}(a_1, b_1)) = \begin{pmatrix} \text{NOR}(-(a_2, b_2)[0], \text{TW}(a_1, b_1)) \\ \text{or } (\text{TW}(-(a_2, b_2)[0], \text{TW}(a_1, b_1)) \\ -(a_2, b_2)[1] \end{pmatrix}$$

对于第 n 位. $\{ -(a_n, b_n) = (\text{NOR}(a_n, b_n), \text{TW}(a_n, b_n)) \quad ①$

$$\begin{aligned} & T(-(a_n, b_n), -(a_{n-1}, b_{n-1})[1]) \\ &= \left(\text{NOR}(-(a_n, b_n)[0], -(a_{n-1}, b_{n-1})[1]), \right. \\ & \quad \left. \text{or}(\text{TW}(-(a_n, b_n)[0], -(a_{n-1}, b_{n-1})[1]), \right. \\ & \quad \left. -(a_n, b_n)[1]) \right) \end{aligned}$$

$n=1$ 时 $-(a_{n-1}, b_{n-1})[1] = 0$

若 $A \geq B \rightarrow i \geq j$

对于 X' , 即 $\overline{x_i x_{i-1} \dots x_1}$, 第 n 位 $x_n = T(-(a_n, b_n), -(a_{n-1}, b_{n-1})[1])$
[0]

若 $A < B$, 即 $T(-(a_j, b_j), -(a_{j+1}, b_{j+1})[1]) = 1$

此时计算结果 $\text{len}(X') = j$, 取相同上

可得 X' 与实际计算结果 $-X$ 有下关系:

$$X + X' = \underbrace{100 \dots 00}_{j \times 0} \quad \text{这个是不是差不多就是补码?}$$

从 X' 翻译回 X : $X = \text{not } X' + 1$
反之亦然

关于这个性质是完全
的性质可以后面再推导

补石组值研究与负数定义.

如题沿用加法时设计最大士安位数来方便对齐计算的思路.

是否可以在减法上也沿用这样的设计.

最大十数位: 9位

例: $\begin{array}{r} \downarrow \\ 10110101 \\ - 00101001 \\ \hline 1 = 11011001 \end{array}$

$$\begin{array}{r} 1 \\ X + X : \quad 11101100 \\ + \quad 00010011 \\ \hline 10000000 \end{array}$$

鄧剛填唐果

$$-x = (4+8+32) - (1+2+16+64)$$

$$\cdot \quad 44-83 = -39 = -100111$$

$$= \quad 000100111$$

可见这个二进制值 x' 首位为 1, 显然不能在原先定义的方法中表示正数.
相反可用于表达负数.

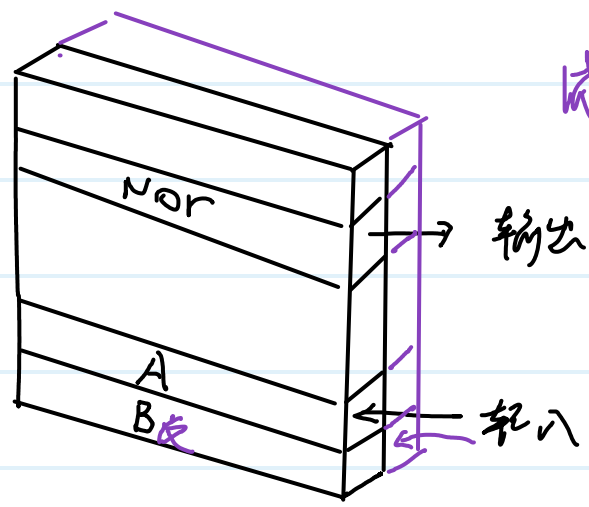
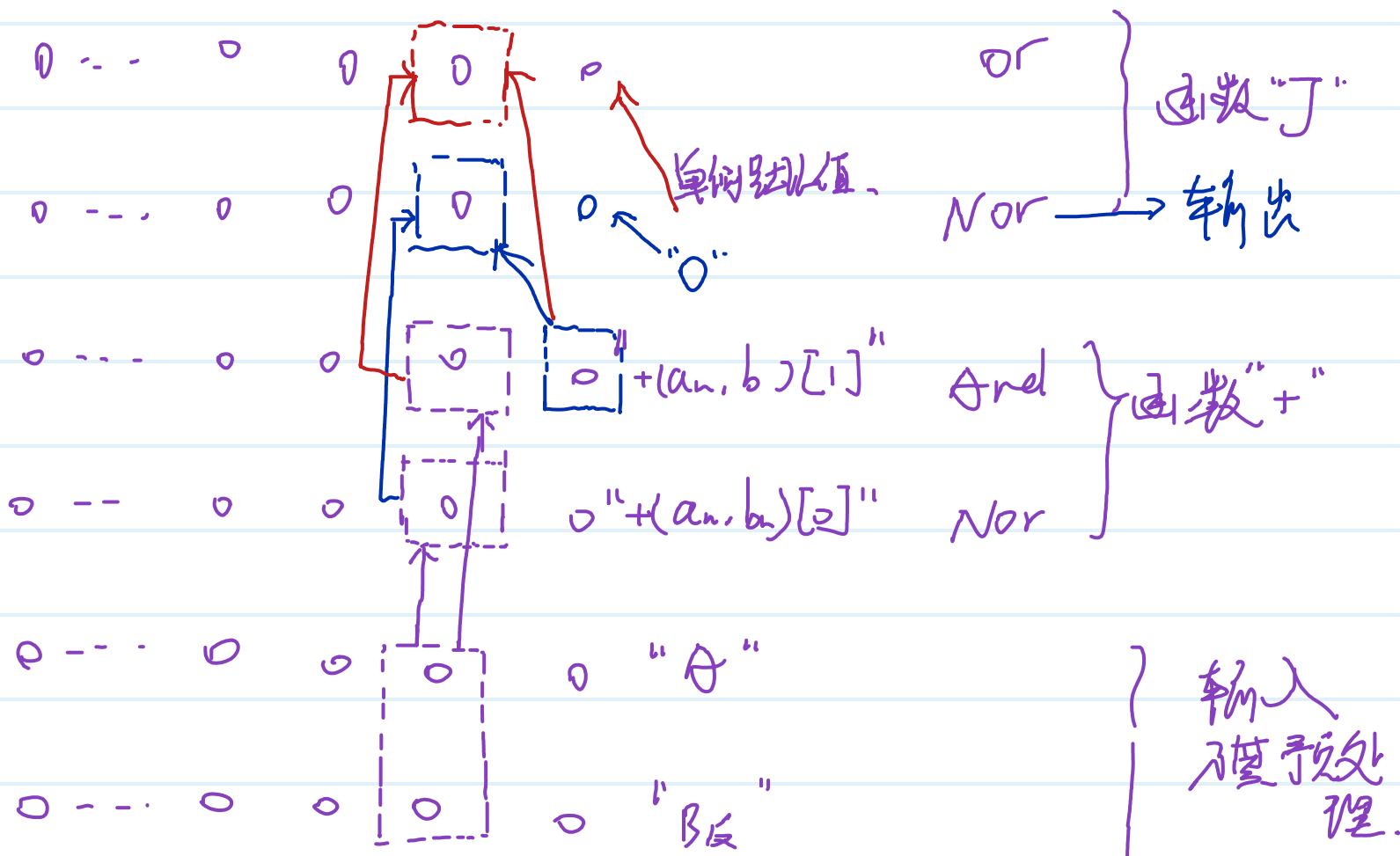
$A-B$ 能否看成 $A+(-B)$, 即一个数先转为反码, 再按加法运算进行。

例: 83-39 (最长9位)

$$\begin{array}{r} 00101011 \\ + 11101100 \\ \hline 10000111 \end{array}$$

$1 \quad 000 \quad 101 \quad 00$
 $\underbrace{\hspace{10em}}_{44}$

更据这个图来设计成块很方便。



测试网同层比加法多了一层反码运算..

$$a_i = b_j = 1, \quad \overbrace{\{a_{i-1} \sim a_1, b_{j-1} \sim b_1\}}^A = \overbrace{\{0, 1\}}^B$$

对于任一二进制加法, $\overbrace{a_i a_{i-1} \dots a_1}^A \cdot \overbrace{b_j b_{j-1} \dots b_1}^B = X$

由上可知对于两个二进制数 M, Z , $\varphi(M, Z) = W$

$$\text{第 } n \text{ 位 } w_n = J(+m_n, z_n, (m_{n-1}, z_{n-1}))$$

[0]

$$\begin{array}{r} \text{破环 } i > j \quad a_i a_{i-1} \dots a_1 \\ \times \quad \quad \quad b_j b_{j-1} \dots b_1 \\ \hline \text{And}(A, b_1) \\ \hline \text{And}(A, b_2) 0 \\ \hline \text{And}(A, b_3) 00 \\ \quad \quad \quad \vdots \\ \hline \text{And}(A, b_j) \underbrace{0 \dots 0}_{(j-1) \times 0} \end{array}$$

安置逻辑门(士兵)和写代码有着很大的差异. 有些用表达式写出来很简单的内容要用物理元件实现比很难.

$$X_1 = \text{And}(A, b_1)$$

$$X_2 = + (X_1, \overline{\text{And}(A, b_2) 0})$$

⋮

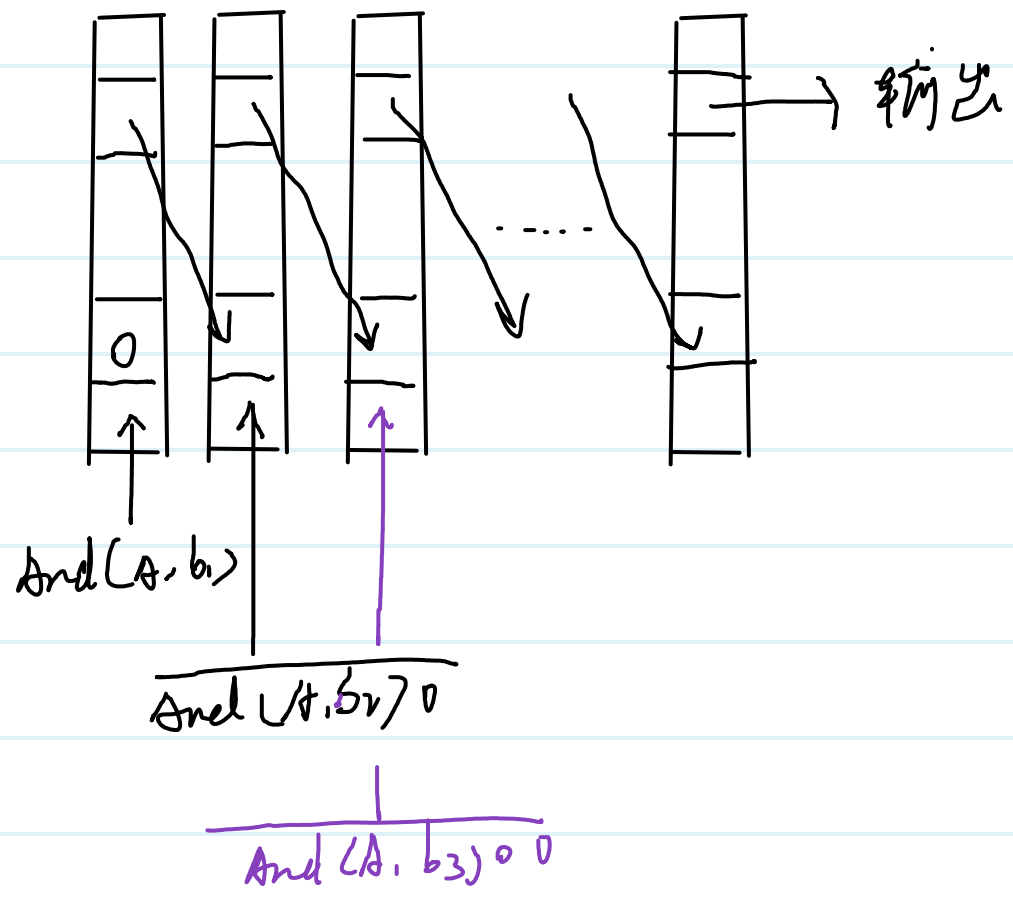
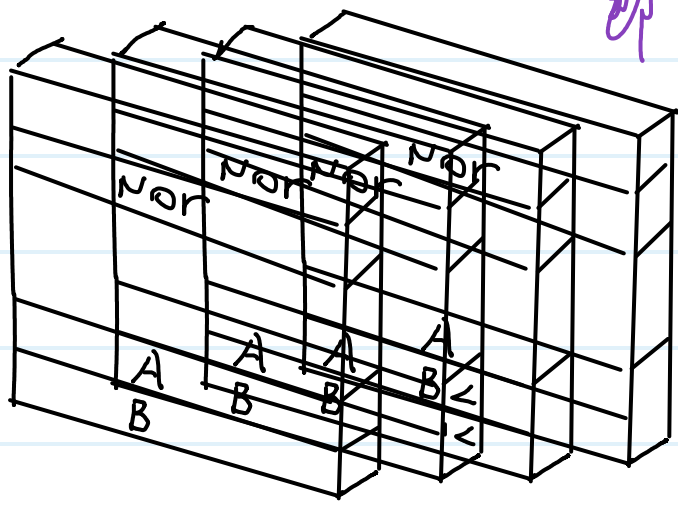
$$X_n = + (X_{n-1}, \overline{\text{And}(A, b_n) \underbrace{0 \dots 0}_{n-1}})$$

⋮

$$X_j = + (X_{j-1}, \overline{\text{And}(A, b_j) \underbrace{0 \dots 0}_{j-1}}) = X$$

Q 前补0和后补0这个操作怎么用逻辑门实现。

A 前补0通过从最长长度实现。后补0通过固定阵列实现。
即需要让傻子士兵负责自顶



除12

不可免有前除除法：余数除法 / 小数除法，这还说不如谁难谁简单
 如果是小数除法，还要考虑能否除尽，即有限小数与无限循环小数的问题。
 还是或者干脆就用分数表示？目前来看，两种表示均有其必要性。

$$a_i = b_j = 1, \quad \left\{ \begin{matrix} a_{i-1} \sim a_1 \\ A \end{matrix} \right., \quad \left\{ \begin{matrix} b_{j-1} \sim b_1 \\ B \end{matrix} \right. = \{0, 1\}$$

对于任一进制加法， $\overbrace{a_i a_{i-1} \dots a_1}^A \div \overbrace{b_j b_{j-1} \dots b_1}^B = X \dots Y$

例如：

$$\begin{array}{r} 000 \mid 101 \\ 101 \overline{) 1010101} \\ \underline{101} \\ 111 \\ \underline{101} \\ 1001 \\ \underline{101} \\ 100 \end{array}$$

战线较长，先用着这个——
 ↑ 与前面计算每一位的“—”有以不同。

参考例子：仅考虑当 $A \geq B$ ，将 A, B 左补0致序列长度为 A', B' ， $\text{len}(A') = \text{len}(B)$
 运算得到 $\text{len}(X') = d$

取 a_d 补全为 $\overbrace{00 \dots 0 a_d}^{(d-1) \text{ 个 } 0}$ ，并计算 $-(\overbrace{00 \dots 0 a_d}^{(d-1) \text{ 个 } 0}, B')[0]$

若 $-(\overbrace{00 \dots 0 a_d}^{(d-1) \text{ 个 } 0}, B')[0] = 1$ ，即 $\overbrace{00 \dots 0 a_d}^{(d-1) \text{ 个 } 0} < B'$ ①
 表示这一位除不，需要向后借位，此时 $\pi_d = 0$

若 $-(\overbrace{00 \dots 0 a_d}^{(d-1) \text{ 个 } 0}, B')[0] = 0$ ，即 $\overbrace{00 \dots 0 a_d}^{(d-1) \text{ 个 } 0} \geq B'$ ②
 表示可以除，无需借位，此时 $\pi_d = 1$

令第 n 次除法产生的中间变量为 $C_n, n \in [1, d]$

$$\text{有 } \pi'_n = \text{not} (-(C_n, B') [0])$$

对于情况 ①,

取 $a_d a_{d+1}$ 补齐为 $\underbrace{00 \dots 0}_{(d-2) \times 0} a_d a_{d+1}$, 同上计算.

对于情况 ②,

取 $\overline{(00 \dots 0 a_d - B')} a_{d+1}$, 同上计算.

$$\text{同上先为 } C_{n-1} = \underbrace{(C_n - \text{And}(B', \pi'_n))}_{\text{这段长度可由总队长中控制}} a_{n-1}$$

综上所述第 n 步 n 取 $d, d-1, \dots, 1$

$$\left\{ \begin{array}{l} \pi'_n = \text{not} (-(C_n, B') [0]) \end{array} \right.$$

$$C_{n-1} = \overline{(C_n - \text{And}(B', \pi'_n))} a_{n-1}$$

$rd = a_d \equiv 0$ 被除数全是正数的情况下.

